LECTURE 5

KNOWLEDGE GRAPHS: FOUNDATIONS FOR CONSEQUENCE-AWARE AGENTS

DATASCI290: NEUROSYMBOLIC AI

JANUARY 21 2026

KNOWLEDGE GRAPHS: LEARNING GOALS

- A clear working definition of a knowledge graph, and the ability to distinguish a KG from “a graph database,” “triples,” and “embeddings-only” representations
  - KG = entities + typed relations + semantics/constraints + provenance/governance
- The ability to explain, using concrete query examples, when a KG is a better fit than a relational database or vector search
  - especially for multi-hop questions, evolving schemas, and explanation-as-path requirements
- A high-level understanding of the two dominant KG paradigms (property graphs and RDF) and what each buys you
  - including why this course uses Neo4j/AuraDB as the pragmatic application-building choice
- The ability to recognize and construct core graph query patterns (even before memorizing syntax)
  - 1-hop neighborhoods, k-hop traversals, and constrained pattern matching
- An understanding of how KGs function inside modern LLM/agent systems as the structured substrate for trust
  - KG-RAG (evidence subgraphs), controlled generation (query-mediated answering), and validation (checking/repair) in high-consequence settings

WHY GRAPHS NOW? (IN THE LLM ERA)

- LLMs are fluent; high-consequence domains need verifiable grounding
  - Missing constraints + invented support are common failure modes
- Knowledge Graphs (KGs) provide explicit **structure**
  - typed relations
  - constraints
  - provenance
  - time (temporal semantics)
- In agents, KGs become the **truth substrate**
  - retrieve evidence subgraph
  - enforce rules
  - verify claims

WHAT IS A KNOWLEDGE GRAPH? (DEFINITION + INTUITION)

- Working definition: *Graph-structured knowledge base with typed entities and typed relations, plus metadata - which is provenance, and time*
- Designed to support
  - traversal
  - reasoning
  - validation
  - auditability
- What it is not
  - Not just a graph database; semantics + IDs + governance matter.
  - Not just embeddings; similarity ≠ meaning/constraints.

DATA MODEL: RELATIONAL VS GRAPH

- The relational model excels when
  - *transactions and aggregations* are the center of gravity
- The graph model excels when
  - *relationships* are the center of gravity (A→B→C paths are common).
- Signature KG questions
  - ‘How is X connected to Y?’
  - ‘What constraints apply given this context?’
  - ‘Show me the evidence trail (subgraph) behind the decision.’

KNOWLEDGE GRAPHS BASICS: WHAT WE MODEL AND HOW WE QUERY

- Reference: Chapter 2 of the Knowledge Graphs texbook
  - Hogaan et al (2021), Knowledge Graphs, Morgan & Claypool
  - https://kgbook.org/
- Our focus is on:
  - KG Modeling
  - KG Querying
- Running example: a tourism board integrating events, venues, routes, cities, and countries.

WHAT DO WE NEED A GRAPH MODEL ANYWAY ?

(WHEN MAY A RELATIONAL MODEL NOT CUT IT ?)

WHY “GRAPH ABSTRACTION” FOR DATA?

- Baseline: a **relational** design
  - Such as Event(name, venue, type, start, end)
- Common modeling pressures that quickly appear:
  - multi-valued attributes (multiple names, multiple venues, multiple types)
  - incomplete/unknown values (future start/end times)
  - schema evolution (new attributes and new entity types as data grows)
  - integration (joining with routes, geography, external sources)
- Graph abstraction: represent data as nodes + edges
  - so that adding new entities/relations is incremental
- Key idea: the “data graph” becomes the substrate, and query languages operate by pattern matching.

DIRECTED EDGE-LABELED GRAPHS (DELG)

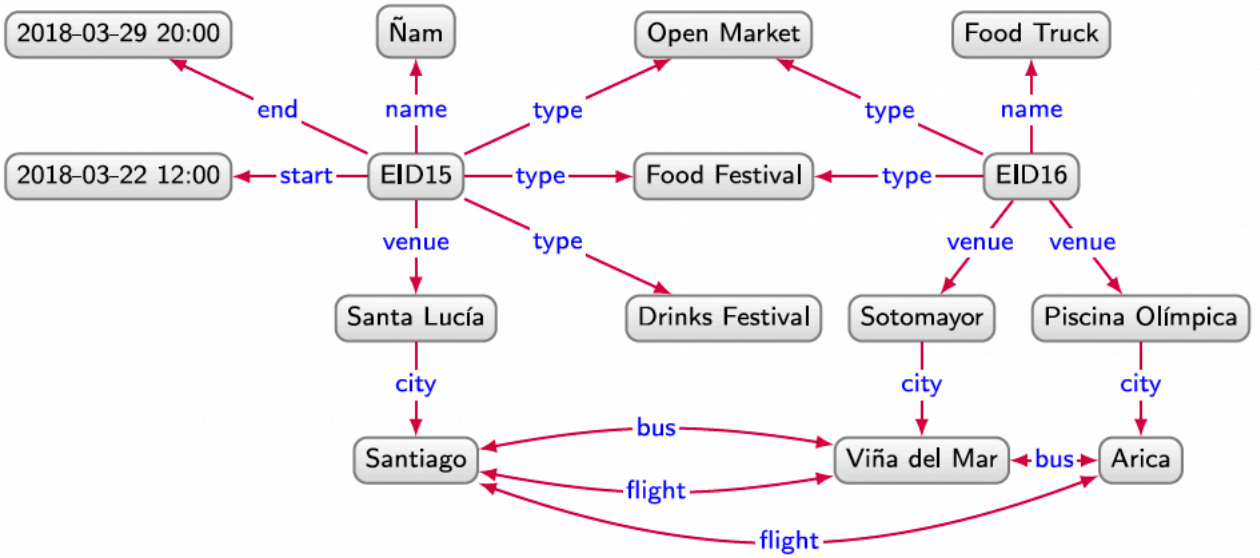
Definition 2.1 (Directed edge-labelled graph)

A directed edge-labelled graph is a tuple $G = (V, E, L)$*, where* $V \subseteq$ **Con** *is a set of nodes,* $L \subseteq$ **Con** *is a set of edge labels, and* $E \subseteq V \times L \times V$ *is a set of edges.*

- Definition: A directed edge-labelled graph G = (V, L, E) where:
  - V = set of nodes (entities or literal values)
  - L = set of edge labels (relation names)
  - E ⊆ V × L × V = set of labelled directed edges (u, ℓ, v)
- Interpretation: each edge is a binary relation instance; multi-valued facts are "just more edges".
- This model underlies "RDF": triples (subject, predicate, object).
- Note:
  - nodes and labels need not be disjoint
  - edges can be missing for some nodes/labels.
- RDF
  - RDF is a W3C-recommended, standardized graph data model: facts are represented as directed, edge-labeled links.
  - RDF nodes can be IRIs (globally unique Web identifiers) and literals (typed values like strings, integers, dates, optionally with language tags).
  - RDF also supports blank nodes: anonymous nodes without identifiers, useful when you don't want to mint internal IDs (e.g., EID15).

DELG EXAMPLE

- Various nodes
- How the tourism-board data becomes edges:
  - Event nodes connect to attributes via labelled edges (e.g., (EID16, type, Food Truck))
  - Routes between cities become edges (e.g., (Arica, flight, Santiago))
  - Incomplete information: simply omit unknown edges (no special NULL required)
- Why it helps: adding a new relation is adding new edges; adding a new entity is adding a node.



HETEROGENEOUS GRAPHS (TYPED NODES)

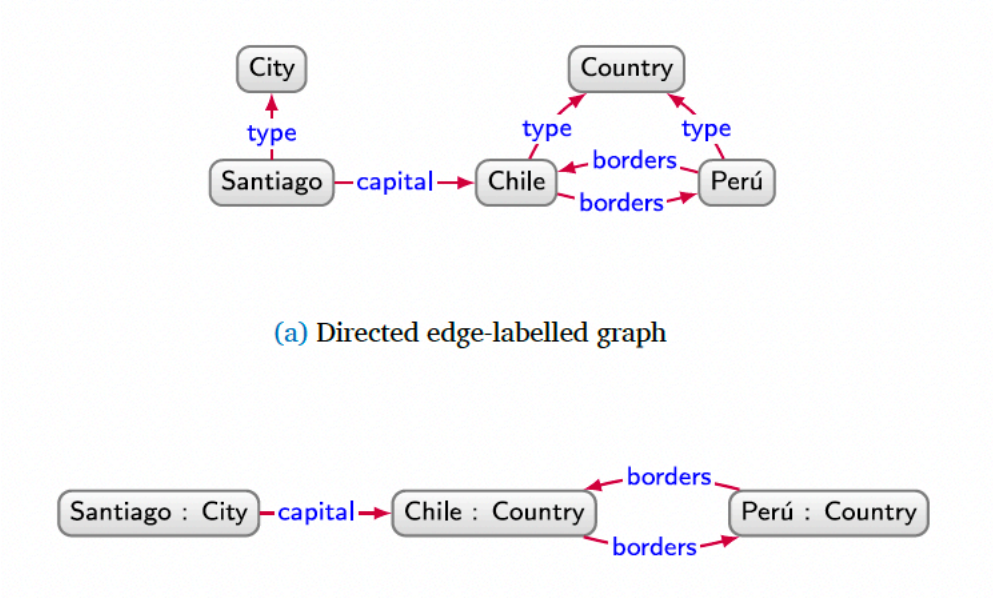
Definition 2.2 (Heterogeneous graph)

A heterogeneous graph is a tuple $G = (V, E, L, l)$*, where* $V \subseteq$ **Con** *is a set of nodes,* $L \subseteq$ **Con** *is a set of edge/node labels,* $E \subseteq V \times L \times V$ *is a set of edges, and* $l : V \to L$ *maps each node to a label.*

- Definition: A heterogeneous graph G = (V, L, E, τ) where:
  - V nodes; L is a shared label/type set; E ⊆ V × L × V edges
  - τ: V → L assigns each node a type (e.g., City, Country)
- Why "heterogeneous": the graph explicitly distinguishes node kinds (and often edge kinds).
- Modeling advantage: type-aware constraints and type-aware querying (e.g., only City —flight→ City).
- Note: types can be rigid if the domain evolves; also, different systems treat "types" differently.

Edge-labelled vs heterogeneous: what changes in practice?

- In edge-labelled graphs: "City" or "Country" are typically represented as edges (rdf:type) or as values.
- In heterogeneous graphs: node labels/types are first-class; systems often exploit them for indexing and validation.
- Key technical tradeoff:
- Edge-labelled graphs unify all facts as triples → very uniform, good for integration.
- Heterogeneous graphs add node typing → can make some traversals and constraints more direct.
- When you need many relationship kinds between the same node pair, edge-labelled graphs handle it naturally.



Property Graphs (IDs, labels, properties)

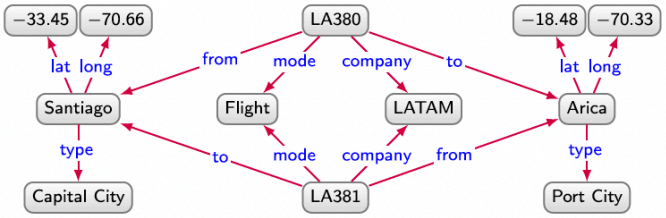
- Definition: A property graph G = (N, E, L, P,Val, h, λ, ρ) where:
- N node IDs, E edge IDs; h: E → N × N gives endpoints
- λ: (N ∪ E) → 2^L assigns a set of labels
- ρ: (N ∪ E) → 2^(P×Val) assigns key–value properties
- Key idea: edges and nodes can both carry properties; edges have identity (edge IDs).
- Simply put: **we can associate property-value pairs**, with nodes or edges !
- Common in systems like Neo4j/Cypher; often optimized for traversals and path-centric queries.

Definition 2.3 (Property graph)

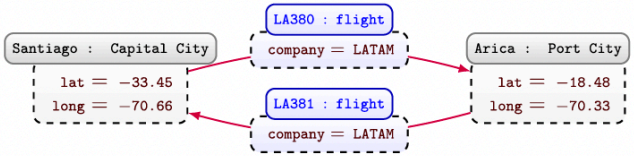
A property graph is a tuple $G = (V, E, L, P, U, e, l, p)$, *where* $V \subseteq$ **Con** *is a set of node ids,* $E \subseteq$ **Con** *is a set of edge ids,* $L \subseteq$ **Con** *is a set of labels,* $P \subseteq$ **Con** *is a set of properties,* $U \subseteq$ **Con** *is a set of values,* $e : E \to V \times V$ *maps an edge id to a pair of node ids,* $l : V \cup E \to 2^L$ *maps a node or edge id to a set of labels, and* $p : V \cup E \to 2^{P \times U}$ *maps a node or edge id to a set of property–value pairs.*

Edge-labeled vs property graph: representational differences

- Edge-labelled graph: every fact is a triple (u, ℓ, v). Adding metadata about an edge usually requires extra nodes/edges.
- Property graph: "attach" metadata directly as properties on nodes/edges (e.g., {start: 2025-01-12}).
- Technical implications:
- Property graphs can represent n-ary/attribute-rich relationships compactly (edge properties).
- RDF/edge-labelled graphs excel at open-world integration and standard exchange formats.
- Query languages differ: SPARQL is pattern/table oriented; Cypher is often path-first.



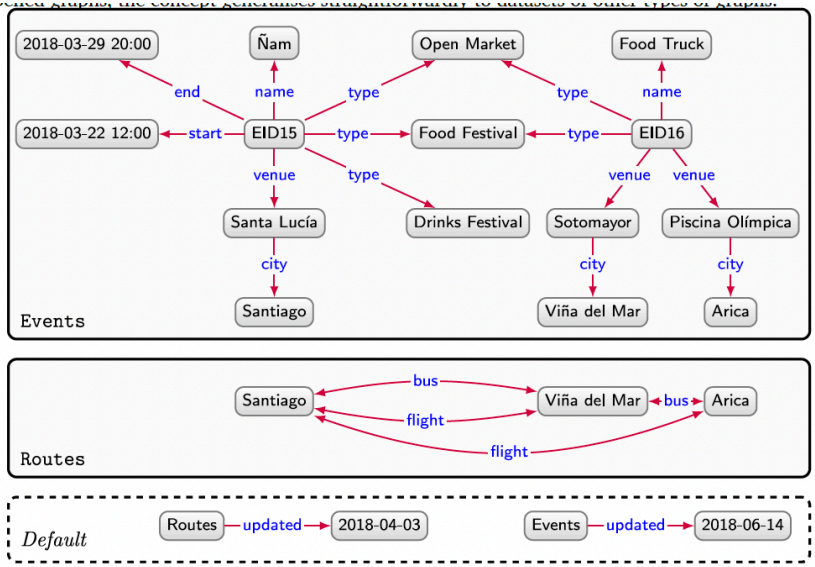
(a) Directed edge-labelled graph



(b) Property graph

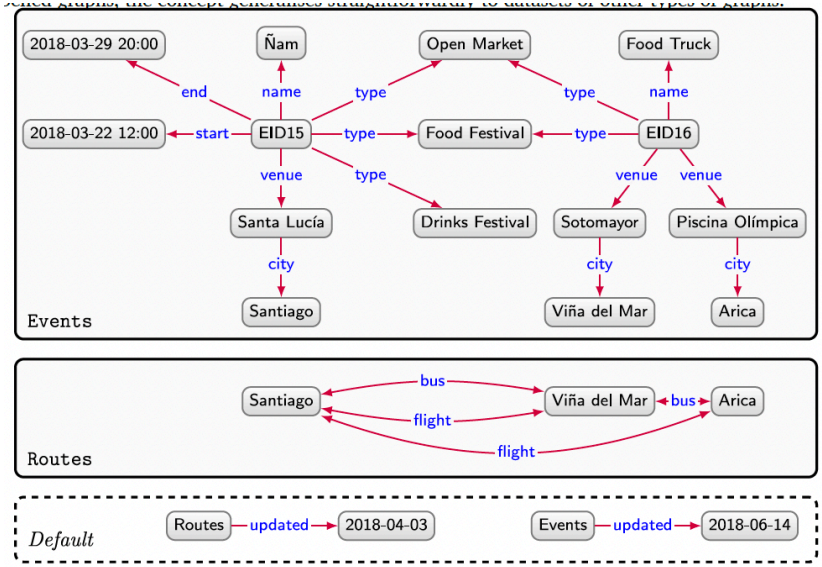
Graph Datasets and Named Graphs

- Why datasets? One project often contains multiple graphs with different provenance or scope.
- Definition:
  - A named graph is (g, n): a data graph g with a graph name n.
  - A graph dataset is (g_0, $\mathcal{G}$): default graph g_0 plus a set of named graphs $\mathcal{G}$ with distinct names.
- Why do we need this ?



Graph Datasets and Named Graphs

- Uses
  - provenance/context (who asserted what, from which source)
  - versioning or staging (draft vs published graph)
  - multi-tenant data separation



Other graph data models (what they buy you)

- Beyond the three core models, systems may use:
- Hypergraphs / n-ary relations: one edge can connect many nodes
  - good for "events" with many participants
- Statement-level metadata: annotate facts with source/time/confidence
  - often via "reification" or edge properties
- Temporal or evolving graphs: edges/nodes valid over intervals; supports time-aware queries.
- Multigraphs: allow parallel edges (same endpoints, different IDs/properties).
- Keep it simple: prefer the simplest model that supports your core queries + integration needs !

Graph stores (where graphs live)

- A graph store is a DBMS designed to persist and query a data graph efficiently.
- Two common ecosystems:
    - RDF stores / triplestores: store triples; expose SPARQL; indexing tuned for pattern joins.
    - Property graph DBs: store nodes/edges with IDs + properties; expose languages like Cypher/ Gremlin.
- Key implementation concerns:
    - Indexing strategies (by subject/predicate/object; by label/property; adjacency lists)
    - Constraints & schema (optional vs enforced; open-world vs closed-world assumptions)
    - Workload shape: join-heavy pattern matching vs traversal-heavy path queries

Querying: from graphs to answers

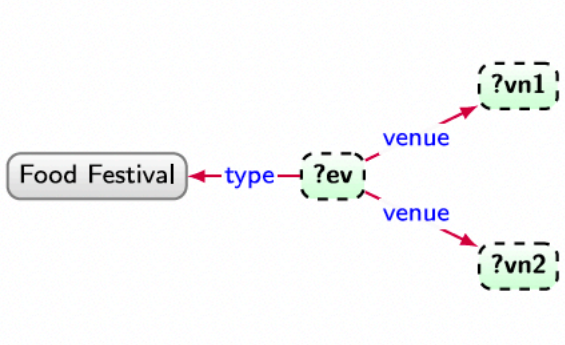
- Core mental model: a query is a "pattern" + operators; evaluation returns a table of bindings.
- Key terms:
- constants: fixed node IDs / labels / property values (e.g., Food Festival)
- variables: placeholders to be bound (e.g., ?event, ?city)
- mapping (binding): a partial function from variables to constants
- A graph pattern matches if there exists a mapping that makes every pattern edge/property true in the graph.

Basic graph patterns (directed edge-labeled)

Definition 2.5 (Basic directed edge-labelled graph pattern)

We define a basic directed edge-labelled graph pattern as a tuple $Q = (V, E, L)$*, where* $V \subseteq$ **Term** *is a set of node terms,* $L \subseteq$ **Term** *is a set of edge terms, and* $E \subseteq V \times L \times V$ *is a set of edges (triple patterns).*

- Definition: a basic directed edge-labelled graph pattern $P = (T_v, T_e, P_e)$
- T_v: node terms (constants or variables)
- T_e: edge-label terms (constants or variables, depending on the language)
- P_e: a set of triple patterns like (s, p, o) where each position is a term
- Shared variables across triple patterns create joins (the main source of expressive power).
- Result is a set of mappings (one per match), often displayed as a table.



?ev	?vn1	?vn2
EID16	Piscina Olímpica	Sotomayor
EID16	Sotomayor	Piscina Olímpica
EID16	Piscina Olímpica	Piscina Olímpica
EID16	Sotomayor	Sotomayor
EID15	Santa Lucía	Santa Lucía

Basic graph patterns (property graphs)

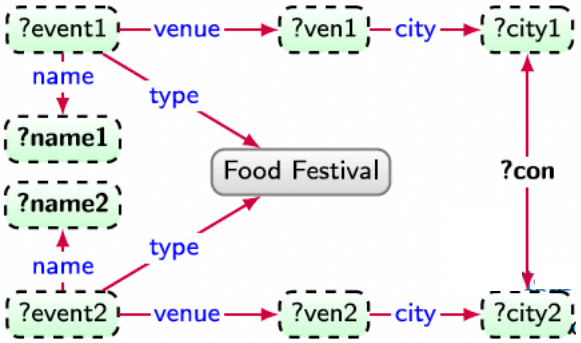
- Definition: introduce **variables** over node IDs, edge IDs, labels, properties, and values.
- Typical pattern ingredients:
  - edge pattern: (a)-[e:Label]->(b) binds endpoints and the edge id
  - label constraints: a has label :City, e has label :flight
  - property constraints: a.country = "Chile" or e.distanceKm > 500
- Compared to RDF basic graph patterns: property graphs usually make “attributes” feel like first-class filters.
- Still the same evaluation idea: find bindings that satisfy all constraints simultaneously.

What does a match “mean”? (evaluation semantics)

- Definition: $[\![P]\!]^G$ is the set of mappings μ that make pattern P true in graph G.
- Two common choices:
- Homomorphism-based: different variables may map to the same constant (more general).
- Isomorphism-based: enforce distinctness (e.g., different variables map to different nodes/edges).
- Practical consequence: homomorphism can yield more answers (and can include “self-joins”).
- Systems may also vary in duplicate handling (set vs bag/multiset) and in NULL/unbound treatment.

Complex graph patterns: building bigger queries

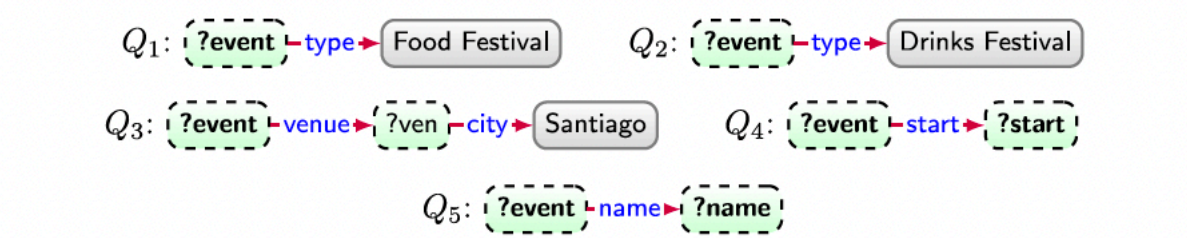
- Idea: basic patterns give a table; relational operators combine/transform those tables.
- Definition: complex graph patterns are built recursively using operators such as:
  - AND / join (⋈): combine matches that agree on shared variables
  - OPTIONAL / left-join (⟕): keep left matches even when right part fails
  - UNION (∪): combine alternative patterns
  - FILTER / selection (σ): restrict matches by conditions (equalities, boolean connectives)
  - Projection (π): keep only selected variables
- Some operators are “syntactic sugar” (do not increase expressivity), others genuinely add power.



?name1	?con	?name2
Food Truck	flight	Ñam
Ñam	bus	Food Truck
Ñam	flight	Food Truck
Ñam	flight	Food Truck

How complex patterns evaluate (relational-algebra view)

- Evaluate a complex pattern by translating it to relational algebra over mapping tables.
- Operational intuition:
  - Join: merge two mapping rows if they are compatible (agree on shared variables).
  - Union: stack answers from two alternatives; duplicates depend on set/bag semantics.
  - Optional (left-join): if right side has no compatible row, keep left row with unbound vars.
  - Filter: drop rows that violate a boolean condition.
- Cyclic patterns are allowed (patterns can “loop back” via shared variables).



$$Q := ((((Q_1 \cup Q_2) \triangleright Q_3) \bowtie Q_4) \bowtie Q_5), \qquad Q(G) =$$

?event	?start	?name
EID16		Food Truck

NAVIGATIONAL GRAPH PATTERNS: VARIABLE-LENGTH REACHABILITY

- Graph query languages often support *path expressions* for traversing arbitrary-length paths
- Path expressions are essentially regular expressions over edge labels:
  - base: an edge label
  - r* (Kleene star): zero-or-more repetitions
  - r1 | r2: disjunction (either)
  - r1 · r2: concatenation (follow one then the other)
  - optionally: r^- (inverse) for 2-way navigation (follow an edge backwards)
- Regular Path Query (RPQ): (x, r, y) asks for paths from x to y whose label sequence matches r.

RDF VS PROPERTY GRAPH: RDF

- RDF / Semantic Web KG (Triples + Ontologies)
  - Core abstraction
    - Facts as triples (subject, predicate, object) with **global identifiers (URIs)**
    - Meaning carried by **ontologies** (RDFS/OWL): classes, properties, domain/range, equivalence, subclass
- Advantages
  - **Interoperability is the default:** URIs + shared vocabularies mean linking across datasets/orgs is native, not an afterthought
  - **Semantics-first inference:** standardized reasoning (subclass/property hierarchies, type entailment) can derive new facts consistently
  - **Decoupled schema evolution:** you can add new predicates/classes without altering rigid table schemas; consumers rely on ontology contracts
  - **Provenance patterns exist at the data-model level:** named graphs / RDF-star support statement-level context
- Query/compute style
  - **SPARQL**: graph pattern matching + joins over triples; great when questions are “find bindings that satisfy this semantic pattern”
- More suited for
  - Multi-institution ecosystems, standards-heavy data: biomed/clinical terminologies, scholarly/citations, gov/linked open data, enterprise metadata catalogs

RDF VS PROPERTY GRAPH: PROPERTY GRAPHS

- Core abstraction
  - Nodes and relationships are first-class, both can carry **properties**; labels/types guide modeling and indexing
- Technical “why it shines”
  - **Traversal is the primitive:** the model + query language are optimized around walking relationships (paths, neighborhoods, subgraphs)
  - **Event and context modeling is natural:** edge properties / event nodes make “this relation with time/ source/confidence” straightforward
  - **Developer ergonomics + operationalization:** Cypher patterns are readable; strong interactive exploration, visualization, constraints, indexes
  - **Great for “application graphs” where the schema iterates:** teams can ship value early and refine without heavy ontology engineering upfront
- Query/compute style
  - **Cypher**: expressive path patterns + constraints; often reads like “draw the graph you want”
- Best suited for
  - Operational and product graphs: fraud/AML, identity & access, recommendations, network/asset graphs, enterprise assistants, agent backends

NEXT LECTURE WE WILL …

- Look more specifically at RDF vs Property Graph representation tradeoffs
  - This is important !
- Revisit our specific interest in Knowledge Graphs
  - From the Trustworthy AI Agents perspective